

# **DETECTING DATA LEAKAGE USING SQL INJECTION**

## **MINI PROJECT REPORT**

*Submitted by*

**DHARSHINI K**

**MONIKA S**

**NIKHILA R S**

*In partial fulfilment for the award of the degree*

*of*

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND BUSINESS SYSTEMS**



**K. RAMAKRISHNAN COLLEGE OF ENGINEERING  
(AUTONOMOUS)  
SAMAYAPURAM, TRICHY – 621112**



**ANNA UNIVERSITY  
CHENNAI – 600 025**

**DECEMBER 2024**

# **DETECTING DATA LEAKAGE USING SQL INJECTION**

## **UCB1512 MINI PROJECT**

*Submitted by*

**DHARSHINI K (8115U22CB016)**

**MONIKA S (8115U22CB037)**

**NIKHILA R S (8115U22CB041)**

*In partial fulfilment for the award of the degree  
of*

**BACHELOR OF TECHNOLOGY**

**IN**

**COMPUTER SCIENCE AND BUSINESS SYSTEMS**

**Under the Guidance of**

**Mrs. M. KARTHIKA, M.E.,**

Department of Computer Science and Business Systems

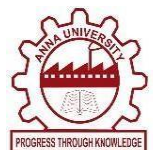
K. RAMAKRISHNAN COLLEGE OF ENGINEERING

**COMPUTER SCIENCE AND BUSINESS SYSTEMS**

**K. RAMAKRISHNAN COLLEGE OF ENGINEERING  
(AUTONOMOUS)**

**Under**

**ANNA UNIVERSITY, CHENNAI**





**K. RAMAKRISHNAN COLLEGE OF ENGINEERING  
(AUTONOMOUS)**



**Under  
ANNA UNIVERSITY, CHENNAI**

**BONAFIDE CERTIFICATE**

Certified that this project report titled “**DETECTING DATA LEAKAGE USING SQL INJECTION**” is the Bonafide work of **DHARSHINI K (8115U22CB016), MONIKA S (8115U22CB037) and NIKHILA R S (8115U22CB041)** who carried out the work under my supervision.

**Dr. J. SASIDEVI, M.E., Ph.D.,  
HEAD OF THE DEPARTMENT  
ASSOCIATE PROFESSOR,  
K. Ramakrishnan College Of  
Engineering (Autonomous)  
Samayapuram, Trichy.**

**Mrs. M. KARTHIKA, M.E.,  
SUPERVISOR  
ASSISTANT PROFESSOR,  
K. Ramakrishnan College Of  
Engineering (Autonomous)  
Samayapuram, Trichy.**

**SIGNATURE OF INTERNAL EXAMINER**

**NAME:**

**DATE:**

**SIGNATURE OF EXTERNAL EXAMINE**



**K. RAMAKRISHNAN COLLEGE OF ENGINEERING  
(AUTONOMOUS)**



**Under  
ANNA UNIVERSITY, CHENNAI**

**ACKNOWLEDGEMENT**

We thank the almighty GOD, without whom it would not have been possible for us to complete our project.

We wish to address our profound gratitude to **Dr. K. RAMAKRISHNAN**, Chairman, K. Ramakrishnan College of Engineering (Autonomous), who encouraged and gave us all help throughout the course.

We express our hearty gratitude and thanks to our honourable and grateful Executive

Director **Dr. S. KUPPUSAMY, B.Sc., MBA., Ph.D.**, K. Ramakrishnan College of Engineering (Autonomous).

We are glad to thank our principal **Dr. D. SRINIVASAN, M.E., Ph.D., FIE., MIW., MISTE., MISAE., C.Engg.**, for giving us permission to carry out this project.

We wish to convey our sincere thanks to **Dr. J. SASIDEVI, M.E., Ph.D.**, Head of the Department, Computer Science and Business Systems Engineering for giving us constants encouragement and advice throughout the course.

We are grateful to **Mrs. M.KARTHIKA, M.E.**, Assistant Professor in the Department of Computer Science and Business Systems, K. Ramakrishnan College of Engineering (Autonomous), for her guidance and valuable suggestions during the course of study.

Finally, we sincerely acknowledge in no less term for all our faculty members, colleagues, our parents and friends for their co-operation and help at various stages in this project work.



**K. RAMAKRISHNAN COLLEGE OF ENGINEERING  
(AUTONOMOUS)**



**Under  
ANNA UNIVERSITY, CHENNAI**

**DECLARATION BY THE CANDIDATE**

I declare that to the best of my knowledge the work reported here in has been composed solely by myself and that it has not been in whole or in part in any previous application for a degree.

Submitted for the mini project Viva-Voce held at K. Ramakrishnan College of Engineering on \_\_\_\_\_

**SIGNATURE OF THE CANDIDATE**

**DHARSHINI K  
(8115U22CB016)**



**K. RAMAKRISHNAN COLLEGE OF ENGINEERING  
(AUTONOMOUS)**



**Under  
ANNA UNIVERSITY, CHENNAI**

**DECLARATION BY THE CANDIDATE**

I declare that to the best of my knowledge the work reported here in has been composed solely by myself and that it has not been in whole or in part in any previous application for a degree.

Submitted for the mini project Viva-Voce held at K. Ramakrishnan College of Engineering on \_\_\_\_\_

**SIGNATURE OF THE CANDIDATE**

**MONIKA S  
(8115U22CB037)**



**K. RAMAKRISHNAN COLLEGE OF ENGINEERING  
(AUTONOMOUS)**



**Under  
ANNA UNIVERSITY, CHENNAI**

**DECLARATION BY THE CANDIDATE**

I declare that to the best of my knowledge the work reported here in has been composed solely by myself and that it has not been in whole or in part in any previous application for a degree.

Submitted for the mini project Viva-Voce held at K. Ramakrishnan College of Engineering on \_\_\_\_\_

**SIGNATURE OF THE CANDIDATE**  
NIKHILA R S  
(8115U22CBO41)

## TABLE OF CONTENT

| CHAPTER NO | TITLE                      | PAGE NO   |
|------------|----------------------------|-----------|
|            | ABSTRACT                   | XI        |
|            | LIST OF FIGURES            | XII       |
| <b>1</b>   | <b>INTRODUCTION</b>        | <b>1</b>  |
|            | 1.1 INTRODUCTION           | 1         |
|            | 1.2 PROBLEM STATEMENT      | 2         |
|            | 1.3 AIM AND OBJECTIVES     | 2         |
|            | 1.4 SCOPE                  | 2         |
| <b>2</b>   | <b>LITERATURE SURVEY</b>   | <b>5</b>  |
| <b>3</b>   | <b>SYSTEM ANALYSIS</b>     | <b>11</b> |
|            | 3.1 EXISTING SYSTEM        | 11        |
|            | 3.2 LIMITATIONS            | 12        |
|            | 3.3 PROPOSED SYSTEM        | 13        |
| <b>4</b>   | <b>SYSTEM REQUIREMENTS</b> | <b>15</b> |
|            | 4.1 HARDWARE REQUIREMENT   | 15        |
|            | 4.2 TECHNOLOGY USED        | 16        |
|            | 4.2.1 SQLITE SOFTWARE      | 16        |
|            | 4.2.2 PYTHON SOFTWARE      | 17        |
|            | 4.2.3 PYTHON MODULES       | 17        |
|            | 4.3 HISTORY OF PYTHON      | 18        |
|            | 4.4 HARDWARE               | 19        |
| <b>5</b>   | <b>MODULE DESCRIPTION</b>  | <b>20</b> |
|            | 5.1 DESCRIPTION            | 20        |
|            | 5.2 USER INPUT VALIDATION  | 21        |
|            | 5.3 SQL QUERY EXECUTION    | 21        |
|            | 5.4 DATABASE INTERACTION   | 22        |



|          |                           |           |
|----------|---------------------------|-----------|
|          | 5.5 LOGGING AND REPORTING | 23        |
|          | 5.6 DETECTION MECHANISM   | 24        |
|          | 5.7 USER INTERFACE        | 25        |
| <b>6</b> | <b>SYSTEM DESIGN</b>      | <b>26</b> |
|          | 6.1 ARCHITECTURE DIAGRAM  | 27        |
|          | 6.2 USE CASE DIAGRAM      | 29        |
|          | 6.3 CLASS DIAGRAM         | 31        |
|          | 6.4 ACTIVITY DIAGRAM      | 32        |
|          | 6.5 DFD DIAGRAM           | 33        |
|          | 6.6 PROCESS SPECIFICATION | 34        |
|          | 6.6.1 ALGORITHM           | 35        |
| <b>7</b> | <b>SYSTEM TESTING</b>     | <b>36</b> |
|          | 7.1 UNIT TESTING          | 36        |
|          | 7.1.1 BACKEND COMPONENTS  | 36        |
|          | 7.1.2 FRONTEND COMPONENTS | 37        |
|          | 7.1.3 UTILITY FUNCTIONS   | 37        |
|          | 7.2 WHITEBOX TESTING      | 37        |
|          | 7.3 BLACK BOX TESTING     | 38        |
| <b>8</b> | <b>CONCLUSION</b>         | <b>39</b> |
|          | <b>APPENDICES</b>         |           |
|          | SAMPLE CODING             |           |
|          | SCREENSHOTS               |           |
|          | <b>REFERENCES</b>         |           |

## **ABSTRACT**

The Aim of the project is to design and detect data leakage using sql injection. The cloud computing holds a massive quantity of private and secret data and information, It uses the internet to communicate with another party so it contain a number of unreliable strings that can be demonstrated to be loopholes, therefore protecting data stored on the cloud is often a serious risk. one of the most serious security dangers to cloud are SQL Injection Attacks. The proposed system detects and prevents unauthorized access the sensitive data through SQL injection attacks. It uses a combination of input validation, parameterized queries, logging, and machine learning-based anomaly detection to identify and respond to potential threats in real-time. The system provides a comprehensive solution for protecting sensitive data and preventing data breaches and leaks. It sends a susceptible query to affect server systems, allowing attackers to gain illegal access to databases, resulting in identity theft and security breaches.

## **LIST OF FIGURES**

| <b>FIGURE NO</b> | <b>FIGURE NAME</b>   | <b>PAGE NO</b> |
|------------------|----------------------|----------------|
| 6.1              | Architecture Diagram | 28             |
| 6.2              | Use Case Diagram     | 30             |
| 6.3              | Class Diagram        | 31             |
| 6.4              | Activity Diagram     | 32             |
| 6.5              | DFD Diagram          | 33             |

# **CHAPTER-1**

## **INTRODUCTION**

### **1.1 Introduction**

Data is crucial for modern businesses. Data leaks sensitive information and data breaches the attacks targeting sensitive data and can lead to significant financial losses and damage a company's reputation. Data breach is an intentional cyberattack where criminals bypass security measures to access sensitive data. SQL Injection attacks pose a major threat to database security by allowing attackers to manipulate SQL queries and gain unauthorized access to data. With the growing reliance on cloud computing platforms, detecting these attacks has become increasingly complex.

As businesses increasingly rely on web applications for data management and customer interactions, the importance of securing these applications against SQL injection attacks cannot be overstated. The consequences of a successful SQL injection attack can be severe, leading to data breaches, financial losses, and significant damage to an organization's reputation.

Therefore, it is crucial to develop effective detection and mitigation strategies to identify SQL injection vulnerabilities and prevent data leakage before it occurs. This project aims to address these challenges by creating a comprehensive framework for detecting SQL injection attempts and safeguarding sensitive information from malicious actors.

## **1.2 Problem Statement**

The increasing frequency and sophistication of SQL injection attacks present a critical challenge to organizations that rely on web applications for data management. Many applications contain vulnerabilities that can be exploited by attackers, leading to unauthorized access to sensitive information and potential data breaches. Despite the availability of various security measures, organizations often struggle to identify and remediate these vulnerabilities effectively, resulting in a heightened risk of data leakage and significant financial and reputational consequences.

The core issue lies in the lack of robust mechanisms for detecting SQL injection attempts in real-time and implementing effective countermeasures. As attackers continuously evolve their techniques, traditional security practices may fall short, leaving applications exposed. Therefore, there is an urgent need for a comprehensive framework that not only detects SQL injection vulnerabilities but also provides actionable insights and strategies to prevent data leakage, ensuring the integrity and confidentiality of sensitive information

### **1.3 Aim And Objective**

The primary aim of this project is to develop a comprehensive framework for detecting SQL injection attacks that can lead to data leakage, thereby enhancing the security posture of web applications. To achieve this aim, the project will focus on several key objectives: identifying common SQL injection vulnerabilities in existing web applications, developing real-time detection mechanisms, implementing effective mitigation strategies, and evaluating the framework's effectiveness against various SQL injection techniques to ensure robust protection against data leakage.

.

### **1.4 Scope**

The scope of this project encompasses a multifaceted approach to addressing SQL injection vulnerabilities and data leakage. It includes conducting thorough research and analysis of SQL injection techniques and their implications for data security, designing and implementing a detection framework that can be integrated into existing web applications, and performing extensive testing using simulated SQL injection attacks to validate the effectiveness of the detection mechanisms. Additionally, the project will provide comprehensive documentation and training materials for developers and security teams, while also identifying areas for future research and development to adapt to evolving threats in the cybersecurity landscape.

The proposed system collects and preprocesses data from web application logs to identify patterns indicative of SQL injection attempts. Machine learning algorithms, such as decision trees, random forests, or neural networks, are trained on labelled datasets to recognize malicious queries. Train various machine learning models using the processed data. Assessing the models using metrics such as accuracy, precision, and recall. And Deploy the best-performing model on a cloud platform for real-time monitoring and detection.

## **CHAPTER-2**

### **LITERATURE REVIEW**

**2.1 TITLE:** A BiLSTM Approach to Enhance the Accuracy of SQL Injection Attack Detection

**AUTHOR:** Junyi Li

**YEAR OF PUBLICATION:** 2024

The study titled "A BiLSTM Approach to Enhance the Accuracy of SQL Injection Attack Detection" by Junyi Li (2024) explores the application of Bidirectional Long Short-Term Memory (BiLSTM) networks in improving the detection of SQL injection attacks. This research highlights the importance of leveraging deep learning techniques to enhance the accuracy of SQL injection detection systems. Presented at the 5th International Conference on Computer Engineering and Application (ICCEA), the study systematically reviews various methods employed in SQL injection detection, emphasizing the advantages of BiLSTM in processing sequential data and capturing contextual information. By analyzing the performance of BiLSTM models against traditional detection methods, the research aims to demonstrate significant improvements in identifying and mitigating SQL injection threats, ultimately contributing to more secure database management practices.



## **2.2 TITLE:** The Model and Method of Information Security Data

Leakage Detection and Prevention

**AUTHOR:** Hao Feng; Hanlin Jiao; Chenyan Zhang

**YEAR OF PUBLICATION:** 2024

The paper by Feng et al. (2024) presents a comprehensive model and methodology for detecting and preventing data leakage in information security systems. It emphasizes the increasing importance of safeguarding sensitive data in an era of rapid digital transformation and cyber threats. The authors review existing techniques for data leakage detection, highlighting their limitations in terms of efficiency and adaptability to evolving security landscapes. They propose an innovative framework that integrates advanced machine learning algorithms with real-time monitoring systems to enhance the accuracy and responsiveness of data leakage prevention mechanisms. By leveraging a combination of behavioral analysis and anomaly detection, the model aims to provide organizations with a robust solution to mitigate risks associated with unauthorized data access and exfiltration. The study underscores the necessity for continuous improvement in security protocols to address the dynamic nature of data threats in contemporary network environments.

## **2.3 TITLE:** SQL Injection Detection using Machine Learning

and Convolutional Neural Networks

**AUTHOR:** Jason Misquitta , S Asha

**YEAR OF PUBLICATION:** 2023

SQL injection (SQLi) remains a critical vulnerability in web applications, prompting a shift from traditional detection methods, such as input validation and web application firewalls, to advanced techniques utilizing machine learning (ML) and deep learning (DL). Research has shown that ML algorithms can enhance detection rates by analyzing patterns in SQL queries, with effective feature extraction methods like TF-IDF playing a crucial role. Convolutional Neural Networks (CNNs) have emerged as particularly promising, achieving high accuracy rates in detecting SQLi due to their ability to automatically learn complex feature representations. Studies, have reported impressive results, with CNN models reaching accuracies exceeding 97.65%. However, challenges such as data imbalance, the need for real-time detection, and the evolving tactics of attackers necessitate ongoing research and model adaptation. Overall, the integration of ML and CNNs in SQL injection detection signifies a significant advancement in cybersecurity, offering robust solutions to protect against one of the most prevalent attack vectors.

**2.4 TITLE:** Research on SQL Vulnerability Mining Based on  
Manual Operation

**AUTHOR:** Guofang Zhang, Qunjun Zheng

**YEAR OF PUBLICATION:** 2023

The study titled "Research on SQL Vulnerability Mining Based on Manual Operation" by Guofang Zhang and Qunjun Zheng (2023) investigates the methodologies for identifying SQL vulnerabilities through manual operations. Presented at the 3rd International Conference on Mobile Networks and Wireless Communications (ICMNWC), this research emphasizes the significance of manual testing techniques in uncovering SQL injection vulnerabilities that automated tools may overlook. The authors discuss various manual operation strategies, including code review and penetration testing, to enhance the detection of SQL vulnerabilities in web applications. By analyzing the effectiveness of these manual approaches, the study aims to provide insights into improving security practices and developing more robust defenses against SQL injection attacks. The findings underscore the importance of combining manual and automated methods to achieve comprehensive vulnerability assessment and strengthen the overall security posture of applications.

**2.5 TITLE:** Research on SQL Vulnerability Mining Based on  
Manual Operation

**AUTHOR:** Guofang Zhang , Quanjun Zheng

**YEAR OF PUBLICATION:** 2023

**SQL Injection Prevention:**

In the study titled "Research on SQL Vulnerability Mining Based on Manual Operation," authored by Guofang Zhang and Quanjun Zheng in 2023, the authors investigate the methodologies for identifying SQL vulnerabilities through manual operations. Presented at the 3rd International Conference on Mobile Networks and Wireless Communications (ICMNBC), this research emphasizes the critical role of manual testing techniques, such as code reviews and penetration testing, in detecting SQL injection vulnerabilities that automated tools may miss. The literature review within the study highlights existing approaches to SQL vulnerability detection, discusses the limitations of automated methods, and advocates for a hybrid approach that combines manual and automated techniques to enhance the overall effectiveness of vulnerability assessment in web applications. The findings aim to contribute to improved security practices and the development of more robust defenses against SQL injection attacks.

## **2.6 TITLE:** Advancements in SQL Injection Attack Detection

**AUTHOR:** Chrisando Ryan Pardomuan

**YEAR OF PUBLICATION:** 2022

The detection of SQL injection attacks using machine learning in cloud computing has garnered significant attention in recent years due to the increasing prevalence of cyber threats targeting web applications. SQL injection, a common attack vector, exploits vulnerabilities in database-driven applications by injecting malicious SQL code, potentially leading to unauthorized data access or manipulation. Traditional detection methods often struggle to keep pace with evolving attack techniques, prompting researchers to explore machine learning (ML) algorithms as a more dynamic and effective solution. Various studies have demonstrated the efficacy of ML models, such as decision trees, support vector machines, and neural networks, in identifying patterns indicative of SQL injection attempts. These models leverage large datasets, including benign and malicious traffic, to train algorithms capable of real-time detection and adaptation to new threats. Additionally, the integration of cloud computing platforms enhances scalability and resource availability, enabling more robust and efficient security solutions. Overall, the intersection of machine learning and cloud computing presents a promising frontier for advancing the detection and mitigation of SQL injection attacks, contributing to the broader field of cybersecurity.

## CHAPTER-3

### SYSTEM ANALYSIS

The process of analysing the system that existed and alterations that are made in the proposed system is stated in system analysis. System analysis is a crucial phase of software development, where we evaluate the current system, identify its weaknesses, and design improvements for a more efficient, secure, and user-friendly solution. In the context of the **"Detecting Data Leaks Using SQL Injection"** project, the system analysis focuses on understanding the limitations of existing SQL injection detection methods, highlighting the shortcomings of current approaches, and proposing a more advanced system with improved functionality.

#### 3.1 EXISTING SYSTEM

The existing systems for detecting SQL injection attacks typically rely on static rule-based systems or web application firewalls (WAFs) that identify known malicious patterns. These systems often use predefined signatures to block common SQL injection techniques, but they are limited in detecting novel or sophisticated attacks that don't match known signatures. Many existing systems lack real-time monitoring or fail to integrate seamlessly with databases, making them less effective at identifying and preventing attacks before they affect the system. While some systems attempt to sanitize user inputs, they may still be vulnerable due to improper configuration or incomplete query parameterization

## 3.2 LIMITATION

The limitations of the existing SQL injection detection systems are varied and pose serious concerns regarding their effectiveness in protecting against modern and evolving attack strategies. Some of the primary limitations include:

1. **Detection Gaps:** Existing systems rely on known attack signatures, which limits their ability to detect new and emerging SQL injection techniques. As attackers become more creative, they can craft injection attacks that do not follow typical patterns, bypassing these detection systems. This gap creates a vulnerability in applications that rely solely on traditional signature-based methods.
2. **False Positives:** Many SQL injection detection systems flag legitimate user inputs or queries as malicious, leading to unnecessary disruptions in service. This can result in legitimate access being blocked, degrading the user experience.
3. **Lack of Real-time Monitoring:** Many existing systems do not provide real-time analysis of incoming queries. As a result, any suspicious activity might go undetected until it has already compromised the database or application. The absence of timely alerts also reduces the system's ability to proactively respond to attacks.

### **3.3 PROPOSED SYSTEM**

The proposed system seeks to address these limitations by integrating both SQL injection prevention and detection mechanisms directly within the application logic and database interactions. Instead of relying solely on signature-based detection, this system incorporates real-time analysis of SQL queries, using Python's built-in `sqlite3` library and regular expressions to identify and block malicious SQL patterns as they occur.

The system uses parameterized queries to prevent SQL injection, ensuring that user inputs are treated as data and not executable code. This approach significantly reduces the risk of injection attacks and eliminates many of the vulnerabilities present in traditional query handling.

The system includes real-time alerts and logging mechanisms, allowing administrators to quickly identify and respond to suspicious queries. It also offers detailed logs for future investigation, enhancing visibility and accountability. This design ensures proactive security, reducing the chances of data breaches and minimizing the impact of any attack attempts.

### **3.4 ADVANTAGES**

The proposed system offers several key advantages over traditional SQL injection detection methods. By utilizing parameterized queries, it effectively mitigates SQL injection risks by ensuring user inputs are treated as data rather than executable code. This eliminates many of the common attack vectors that other systems fail to address.



The system's real-time detection of malicious queries provides immediate alerts, enabling proactive responses to potential security threats. With SQLite integration, the system offers a lightweight, easily deployable solution for both small and large-scale applications, making it scalable and adaptable to various environments. Furthermore, the proposed system's modular design allows for future enhancements, such as integrating machine learning-based anomaly detection for more sophisticated threat detection.

## **CHAPTER-4**

### **SYSTEM REQUIREMENTS**

A The system requirements are designed to ensure smooth development, deployment, and operation of the project, making it secure, efficient, and scalable. On the hardware side, the project necessitates modest computing resources, allowing it to run on most modern systems with minimal overhead.

On the software side, the project utilizes Python as the primary programming language due to its simplicity, cross-platform compatibility, and extensive library support. SQLite is employed as the database solution, offering a lightweight and self-contained environment for data storage and query execution. The Python libraries like sqlite3 for database management and re for pattern matching enhance the system's ability to detect and prevent SQL injection attacks in real time. This combination of minimal hardware requirements and robust software technologies ensures an accessible and effective solution for securing applications against SQL injection vulnerabilities.

#### **4.1. HARDWARE REQUIREMENTS**

|           |   |               |
|-----------|---|---------------|
| Hard Disk | : | 600GB         |
| RAM       | : | 4 GB          |
| Processor | : | Intel Core i5 |

## **SOFTWARE REQUIREMENTS**

|                       |                                 |
|-----------------------|---------------------------------|
| Operating System      | : Windows 10 or Linux or ubuntu |
| User Interface        | : CLI (command line interface)  |
| Client-side Scripting | : HTML                          |
| Programming Language  | : Python                        |
| Database              | : SQLite                        |

## **4.2 TECHNOLOGIES USED**

### **4.2.1 SQLite**

SQLite is a lightweight, serverless, and self-contained relational database management system that is widely used for local data storage in applications. It is particularly suitable for projects involving data leakage detection due to its simplicity and ease of integration with Python through libraries like `sqlite3`.

In this data leakage detection project, SQLite can be utilized to store and manage datasets, logs, and results of data analysis efficiently. Its ability to handle complex queries allows for effective data manipulation and retrieval, making it an ideal choice for analysing patterns and anomalies in data that may indicate leakage. Additionally, SQLite's cross-platform compatibility ensures that the project can be easily deployed across different environments, enhancing its accessibility and usability.

## **4.2.2 PYTHON**

Python provides a robust ecosystem for database interaction, making it an excellent choice for projects focused on detecting data leakage. With libraries such as `sqlite3`, and `pandas`, Python enables seamless connectivity to various database systems, including SQLite, PostgreSQL, and MySQL.

In this data leakage detection project, Python can be employed to efficiently store, query, and analyse large datasets, allowing for the identification of patterns and anomalies that may signify data breaches. The language's rich set of data manipulation tools, combined with its ability to execute complex SQL queries, facilitates thorough investigations into data integrity and security.

Furthermore, Python's versatility and ease of use empower developers to implement machine learning algorithms and statistical methods to enhance the detection process, making it a powerful tool in safeguarding sensitive information.

## **4.2.3 PYTHON MODULES AND LIBRARIES**

Python libraries play a crucial role in the project's implementation. The `sqlite3` library is used for establishing secure database connections and performing query operations. It ensures safe interaction with the SQLite database through parameterized queries, effectively mitigating injection risks. The `re` library is employed to detect suspicious patterns in user inputs using regular expressions, enhancing the system's ability to identify malicious activity in real-time.

### 4.3. HISTORY OF PYTHON

Python, created by Guido van Rossum, was first released in 1991 as a high-level, interpreted programming language designed with a focus on readability and simplicity. Van Rossum envisioned a language that emphasized code readability and allowed programmers to express concepts in fewer lines of code than traditional programming languages like C. Python's early development was influenced by the ABC programming language, which was also developed by Van Rossum and aimed at teaching programming. Python was designed to be intuitive, with a clean syntax that mimicked natural language, making it accessible to both beginners and experienced developers alike.

In the years following its release, Python underwent continuous improvement. Python 2.0, which debuted in 2000, introduced several new features such as garbage collection, list comprehensions, and improved support for Unicode, which enhanced the language's usability. Python 2 became the dominant version for many years, supported by a large community and a growing number of libraries and frameworks. However, in 2008, the Python Software Foundation announced Python 3.0, a major redesign of the language aimed at removing redundancy and improving consistency. Python 3 introduced many changes, such as a more efficient handling of strings and changes to syntax, though it was not backward compatible with Python 2, causing some resistance during the transition period.

By the 2010s, Python had become one of the most popular programming languages in the world. Its versatility, combined with a rich ecosystem of libraries and frameworks, led to widespread adoption in diverse fields such as web development, data science, machine learning, artificial intelligence, automation, and scientific computing. As of 2020, Python 3 became the standard version, and Python 2 reached its end-of-life in January 2020. Today, Python continues to evolve with regular updates, and its simple, powerful syntax and vast array of applications ensure its place as one of the most influential programming languages in the world.

### **4.3 Hardware**

In The hardware requirements for the "Detecting Data Leaks Using SQL Injection" project are minimal, as it is designed to run on standard personal computers. The primary hardware used includes a basic system with a processor such as an Intel Core i3 or equivalent, though an Intel Core i5 or higher is recommended for smoother performance, especially when handling more complex queries or larger datasets. A minimum of 4 GB of RAM is required, with 8 GB or more being preferable for optimal operation, as it ensures smooth multitasking and faster processing when managing multiple inputs or simultaneous operations.

In terms of storage, the project requires at least 20 GB of free space on the hard drive for installation, database storage, and logs, with 50 GB being ideal for future scaling and additional data storage needs. The system should also have a reliable internet connection for accessing external resources, libraries, and updates, although the project itself does not rely on external server setups.

## **CHAPTER-5**

### **MODULE DESCRIPTION**

#### **5.1 Description**

The Detecting Data Leaks Using SQL Injection project is divided into several key modules that work together to ensure secure handling of user inputs and protect against SQL injection attacks. The User Input Validation Module checks incoming data for malicious patterns using regular expressions. The SQL Query Execution Module safely executes SQL queries by utilizing parameterized queries to prevent injection.

The Database Interaction Module manages database connections and handles secure data retrieval and updates. The Logging and Reporting Module records all system activities and detects suspicious actions, providing alerts for potential threats. The Detection and Prevention Mechanism Module analyses inputs for common injection techniques and blocks harmful data before it reaches the database. Optionally, a User Interface Module provides an interactive front-end for user interactions in a web-based or GUI-based setup. Each module plays a crucial role in ensuring the security, efficiency, and effectiveness of the project in preventing data leaks caused by SQL injection.

## **5.2 User Input Validation Module**

The User Input Validation Module is responsible for ensuring that all user inputs are thoroughly vetted before being processed by the system. It plays a crucial role in preventing SQL injection attacks by inspecting user-supplied data for any suspicious or potentially harmful patterns. This is achieved using Python's re (regular expression) module, which checks the input for SQL-specific keywords, special characters, or injection tactics such as the use of semicolons, single quotes, or comments that could alter the intended query execution. If any input is deemed suspicious or harmful, it is rejected or sanitized, ensuring that only safe data reaches the SQL query execution phase.

This module ensures that inputs are properly formatted and adhere to expected types or structures. It serves as the first line of defence in maintaining system security by preventing common attack vectors from gaining access to the database. For instance, fields that should contain only numeric data are checked to ensure they do not contain alphabetic characters or SQL syntax, thus minimizing the risk of malicious code being executed within the database.

## **5.3 SQL Query Execution Module**

The SQL Query Execution Module handles the safe execution of SQL queries after user inputs have passed through the validation phase. This module directly interacts with the SQLite database, executing queries while ensuring that user inputs do not interfere with the integrity of the SQL statements.



To mitigate the risk of SQL injection, the module relies heavily on parameterized queries, a method that ensures user inputs are treated as data rather than executable code. By binding user input as variables in the query, this approach prevents malicious users from injecting harmful SQL code that could potentially compromise the database. It is also responsible for managing the overall flow of database operations such as data retrieval, insertion, and updating. The secure execution of queries ensures that the system remains robust even when handling potentially complex or large datasets.

It also includes error handling mechanisms that gracefully manage any issues that might arise during query execution, providing feedback to the user without exposing sensitive system details.

## **5.4 Database Interaction Module**

The Database Interaction Module manages the connection between the system and the SQLite database, ensuring secure and efficient communication between them. This module handles establishing, maintaining, and closing database connections as needed. It also abstracts the complexity of database interactions, providing a clear interface for executing SQL queries. The module uses Python's `sqlite3` library to connect to the SQLite database, and it plays an essential role in preventing unauthorized access by enforcing proper security measures and query sanitization practices.

Beyond basic query execution, this module is designed to handle exceptions and errors that may occur during database interaction. For instance, it manages connection issues or incorrect SQL syntax gracefully, ensuring the system remains stable. It also supports functionalities such as transaction management, ensuring that database changes are committed safely or rolled back in case of failures. Overall, this module is crucial for the system's robustness, offering a secure, reliable means of interacting with the underlying database.

## **5.5 Logging And Reporting Module**

The Logging and Reporting Module is responsible for maintaining detailed logs of all system activities, including user inputs, executed queries, and detected security threats. The logging library in Python is utilized to create a structured logging system that tracks each step of the process, providing transparency and accountability. This module records every instance of data interaction, including successful queries and any attempts at SQL injection, allowing administrators to review the logs for signs of malicious behaviour. By capturing a history of interactions, it enables a post-attack analysis and helps improve the system's security.

The reporting function allows the system to generate alerts whenever suspicious activity is detected, such as the use of potentially harmful input patterns or SQL injection attempts. These alerts can be configured to notify system administrators in real-time, allowing them to take immediate action if necessary.

The logging module thus plays a dual role: it not only helps in tracking and debugging issues but also serves as a critical component in identifying and mitigating potential security threats.

## **5.6 Detection Mechanism Module**

The Detection Module is the core security component of the system, designed to identify and neutralize SQL injection attempts in real-time. It continuously analyses incoming user data, using regular expressions to detect known malicious patterns or behaviours associated with SQL injection attacks. The module looks for signs such as the inclusion of SQL keywords, syntax manipulations, or unusual characters that may indicate an attempt to exploit vulnerabilities in SQL queries. If any malicious input is detected, the system either blocks the request or sanitizes it to neutralize the threat.

This module also integrates with the rest of the system to prevent data leaks and unauthorized access. When an attack is detected, it triggers preventive actions such as rejecting the query or notifying the administrator, ensuring that the system remains secure. The mechanism also adapts over time by incorporating new threat patterns and improving the system's ability to detect sophisticated attacks, making it a dynamic defence against evolving security threats.

## 5.6 User Interface Module

The primary version of the project runs as a command-line interface (CLI), the User Interface (UI) Module is an optional extension for those wishing to interact with the system through a web interface or graphical user interface (GUI). In this extended version, the UI module provides an interactive platform where users can submit queries or data inputs, view results, and receive alerts regarding potential SQL injection attempts. The module is responsible for handling user interactions and displaying outputs in a user-friendly format.

In a web-based or GUI-based application, frameworks such as Flask or Django could be used to build the UI, making it easy for users to interact with the system through forms, buttons, and visual cues. This module bridges the gap between the end user and the underlying detection and prevention mechanisms, ensuring a seamless and intuitive experience while maintaining the same level of security and functionality offered by the backend modules.

## **Chapter-6**

### **System Design**

The system design of the "Detecting Data Leaks Using SQL Injection" project follows a modular architecture aimed at ensuring secure, efficient, and scalable detection and prevention of SQL injection attacks. The core of the design is based on input validation, secure query execution, and database interaction, structured in separate layers to ensure clean separation of concerns. The first layer of the system deals with capturing and validating user inputs. This layer ensures that data entered by users is safe by checking for malicious SQL patterns using regular expressions. Once validated, the inputs are passed to the query execution module, which safely interacts with the SQLite database using parameterized queries to prevent any SQL injection vulnerabilities.

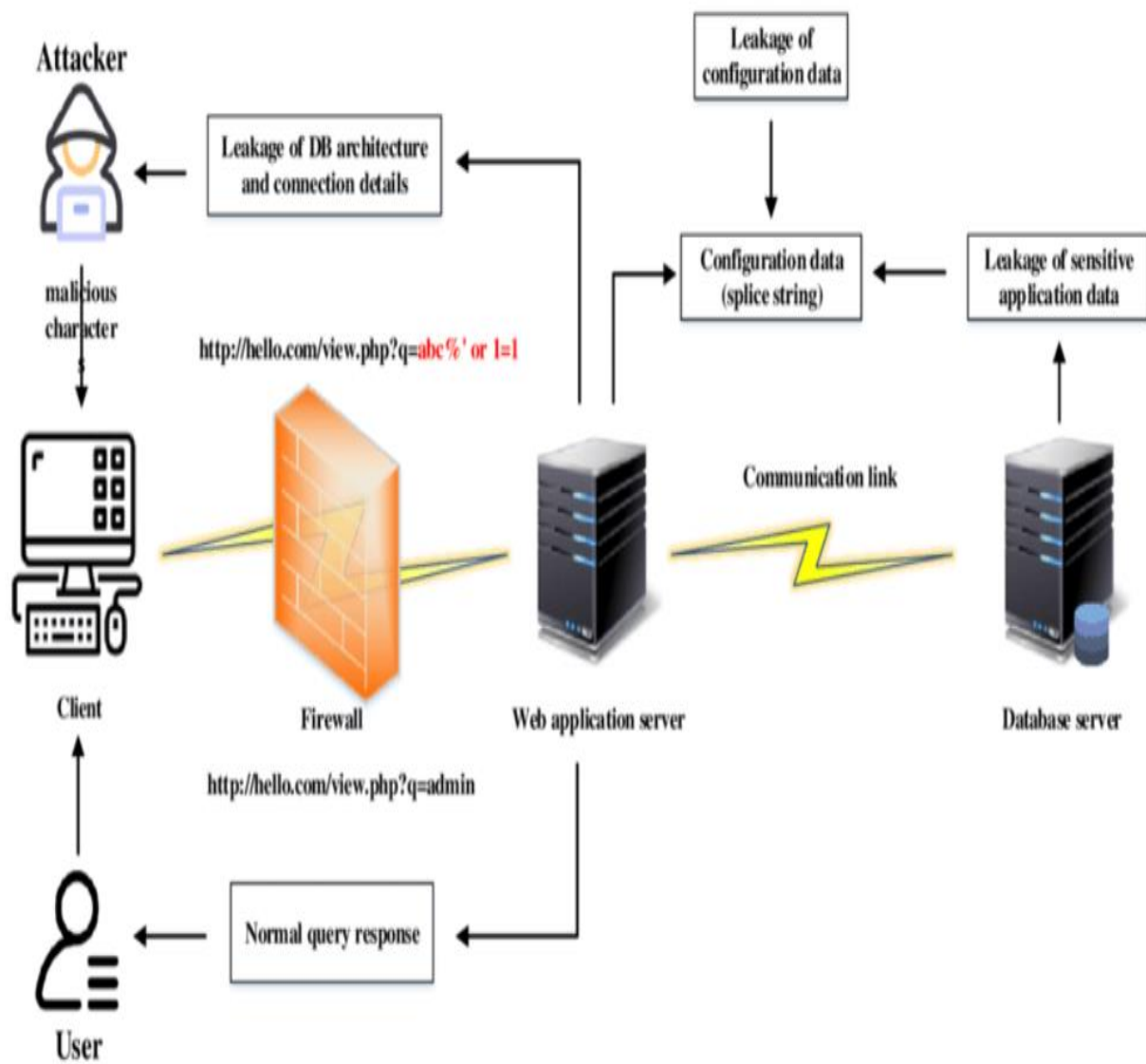
The database interaction layer ensures secure communication with the SQLite database, handling query execution and error management. It ensures that all database operations are performed without exposing the system to risks associated with unsafe SQL queries. A logging and reporting mechanism is integrated into the system design, which records all queries and detected security events. This log acts as both an audit trail and a diagnostic tool for administrators, helping track any suspicious activity and providing insights into potential security threats.

The design also accommodates future extensibility, this flexibility ensures that as the threat landscape evolves, the system can adapt and incorporate advanced methods to stay effective against emerging SQL injection tactics. The overall design prioritizes security, user experience, and performance, ensuring that the system remains robust while providing a simple interface for interaction.

## **6.1 Architecture diagram**

The software architecture diagram is a visual presentation of all of the aspects that constitute a system, either in part or whole. It is a depiction of a set of concepts that comprise architecture, such as its principles, components, and materials. It is also a system diagram used to abstract the general layout of the software system as well as the interactions, limitations, and limits between parts.

Also, an architecture diagram is a network map used to describe the general structure of a software program as well as the interactions, restrictions, and limits between elements. It is a significant tool since it offers a broader picture of the computer underlying physical installation as well as its development plan.



**Figure 6.1: Detect data leakage [Architecture]**

## 6.2 Use Case Diagram

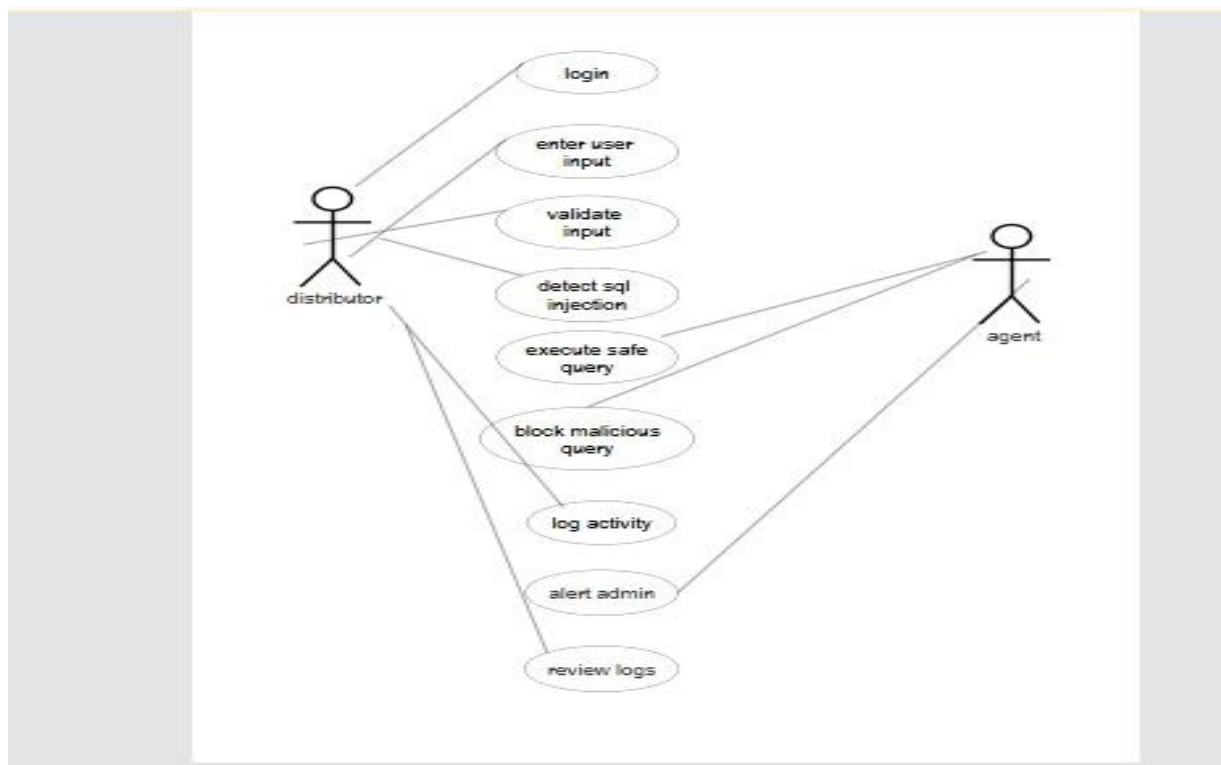
The description enables to have a proper understanding of how actor interacts with the

system through their use cases

| ACTOR  | USE CASE              | USE CASE DESCRIPTION                                                                                                                           |
|--------|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Users  | Enter User Input      | The user submits data to the system through a form.                                                                                            |
| System | Validate Input        | The system checks the submitted input to ensure that it is formatted correctly and does not contain malicious SQL code.                        |
|        | Detect SQL Injection  | The system analyses the user input for patterns indicative of SQL injection attacks using regular expressions or predefined attack signatures. |
|        | Execute Safe Query    | If the input is determined to be safe, the system executes the SQL query on the database using parameterized queries to prevent SQL injection. |
|        | Block Malicious Query | If the input contains SQL injection patterns, the system blocks the query from being executed and flags it as malicious.                       |



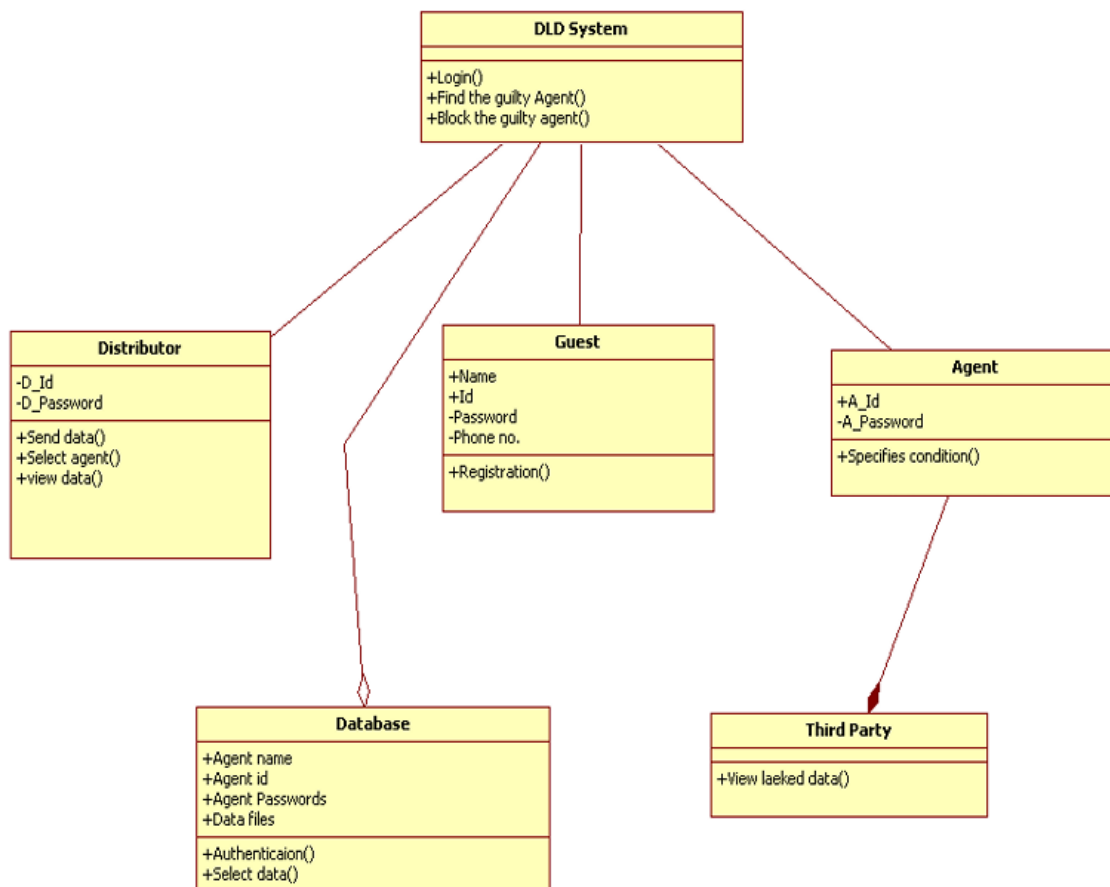
|       |                      |                                                                                                                                                            |
|-------|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|
|       | Log Activity         | The system logs all user activities for auditing and further analysis, including submitted queries, detected attacks, and normal user interactions.        |
|       | Alert Admin          | If the system detects a suspicious query or a potential SQL injection attack, it sends an alert to the admin for immediate review and action.              |
|       | Prevent Data Leakage | The system prevents sensitive data from being exposed during a potential SQL injection attack by blocking malicious queries and protecting data integrity. |
| Admin | Review Logs          | The admin reviews the system logs to investigate the cause of suspicious activities or attacks and decides on corrective actions or further monitoring.    |



**Figure 6.2: Detect data leakage [Use case Diagram]**

## 6.3 Class Diagram

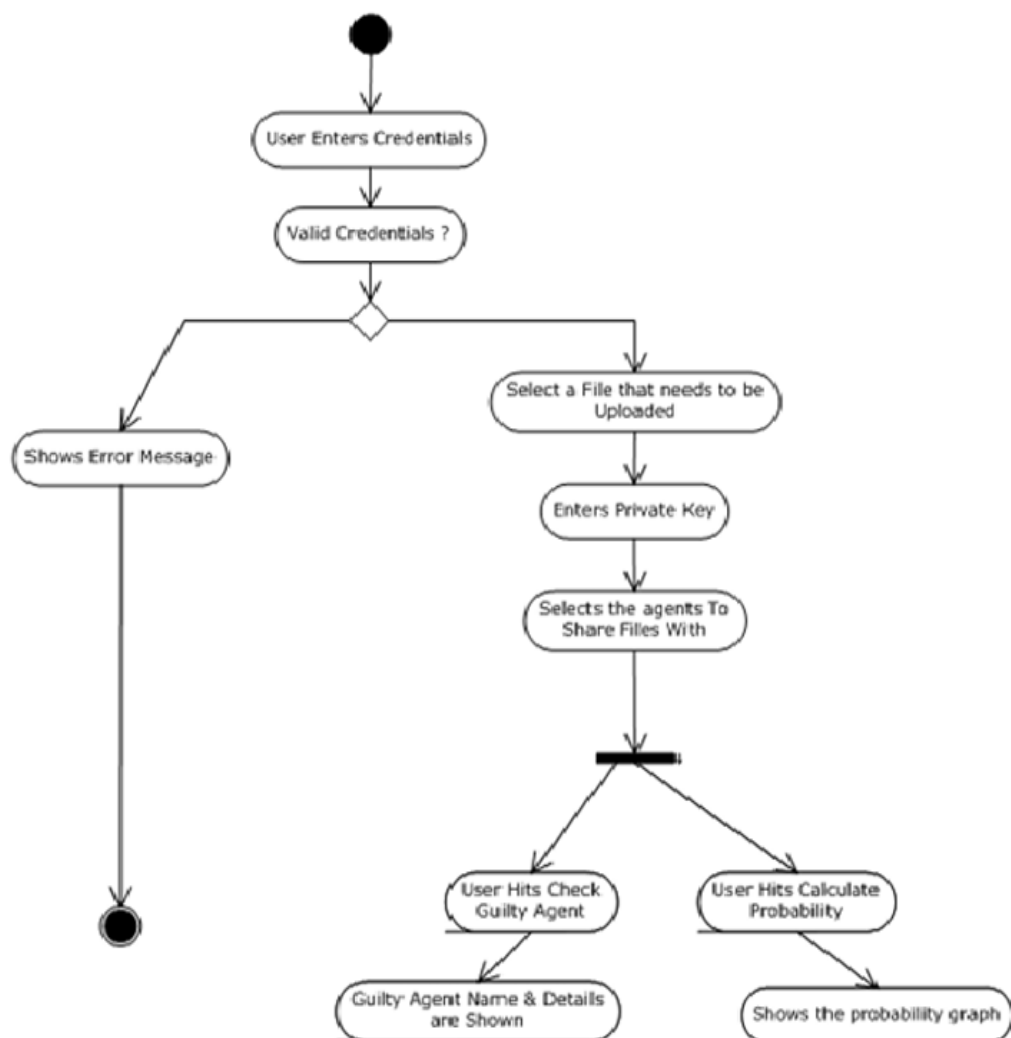
A class diagram provides a static view of the system, showcasing the classes involved in the detection of SQL injection and their relationships. The class diagram can represent the components of the application that interact with the database and handle user input.



**Figure 6.3: Detect data leakage [Class Diagram]**

## 6.4 Activity Diagram

An activity diagram visually represents the workflow involved in this detection process, outlining the sequence of actions and decisions. It begins with user input, followed by input validation and sanitization to ensure conformity to expected formats. The next steps involve checking for vulnerabilities in the application, implementing prepared statements or parameterized queries to prevent SQL injection, and logging all executed SQL queries for monitoring purposes.

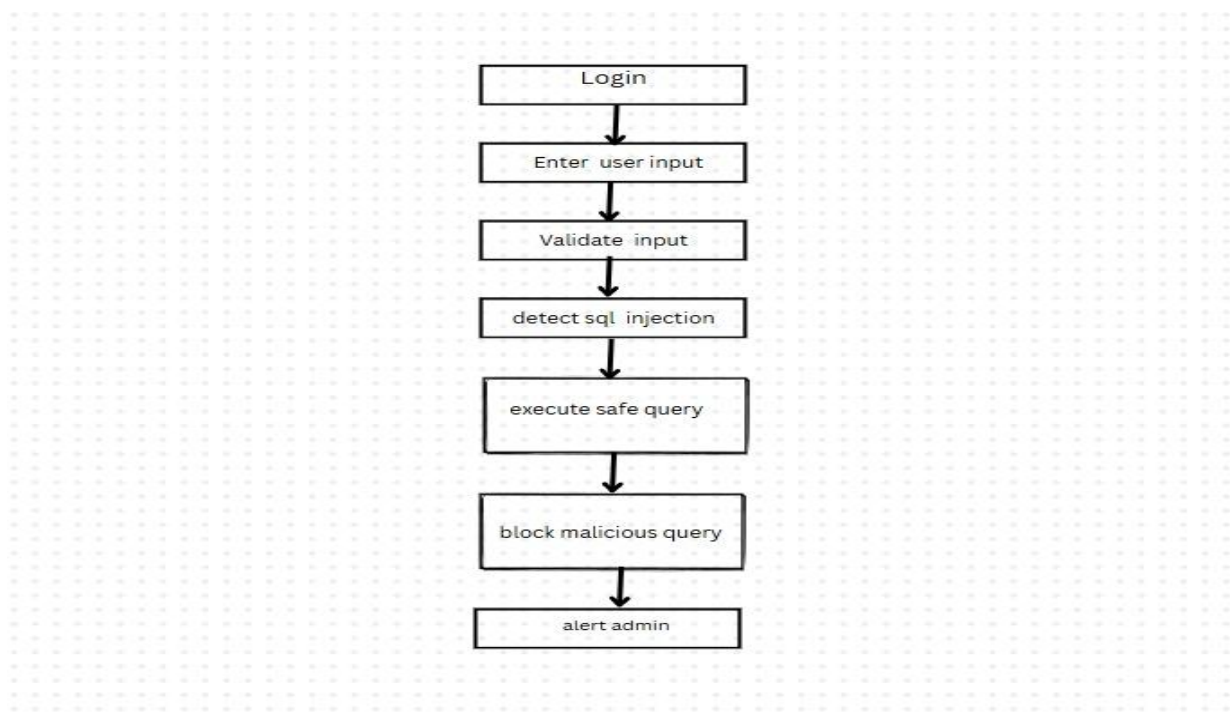


**Figure 6.4: Activity Diagram**

## 6.5 DFD Diagram

A Data-Flow Diagram (DFD) is a graphical representation of the "flow" of data through an information system. DFDs can also be used for the visualization of data processing

(structured design). On a DFD, data items flow from an external data source or an internal data store to an internal data store or an external data sink, via an internal process. Dataflow diagrams are an intuitive way of showing how data is processed by a system. At the analysis level, they should be used to model the way in which data is processed in the existing system. The notations used in these models represent functional processing, data stores and data movements between functions. Data flow models are used to show how data flows through a sequence of processing steps.



**Figure 6.5: DFD diagram**

## 6.6 Process Specification

In a detecting data leakage through SQL injection involves a systematic approach that begins with understanding the nature of SQL injection attacks, which exploit vulnerabilities in applications by manipulating SQL queries.

The first step is to identify potential points of vulnerability, such as user input fields and dynamic SQL query constructions. To mitigate these risks, it is essential to implement security measures like using prepared statements or parameterized queries, validating and sanitizing user inputs, and ensuring robust error handling to prevent the exposure of sensitive database errors. Throughout monitoring and logging SQL queries executed by the application can help identify unusual access patterns indicative of potential attacks. Regular automated testing, including penetration testing and static code analysis, is crucial for uncovering vulnerabilities.

In the event of a detected incident, having an alerting system for suspicious activities and a data breach response plan is vital for timely action. By adhering to these practices, organizations can effectively detect and prevent data leakage resulting from SQL injection attacks, thereby safeguarding sensitive information.

.

## Algorithm

```
SELECT * FROM users WHERE username = " OR '1'=1';
```

```
UNION SELECT password FROM users WHERE 'x'='x';
```

```
SELECT query_id, user_id, ip_address, query_text, execution_time
```

```
FROM
```

```
    query_logs
```

```
WHERE
```

```
    query_text LIKE '%UNION%'
```

```
    OR query_text LIKE '%SELECT%'
```

```
    OR query_text LIKE '%--%'
```

```
    OR query_text LIKE '%;%'
```

```
    OR query_text LIKE '%DROP%'
```

```
    OR query_text LIKE '%OR 1=1%'
```

```
    OR LENGTH(query_text) > 1000; -- Abnormally long queries
```

## CHAPTER-7

### SYSTEM TESTING

#### 7.1 Test Cases for Unit Testing

In Unit testing in this project focuses on testing individual modules to ensure that each component functions as expected. The primary goal of unit testing is to verify the correctness of specific functions and methods within the system. The SQL Query Execution Module is tested by executing parameterized queries with different types of user data, ensuring that they are securely executed without triggering SQL injection vulnerabilities. Unit tests are written to cover each function, ensuring that each component can work independently without errors. Unit tests are executed in isolation from other parts of the system to confirm that the individual modules are performing their intended tasks.

##### 7.1.1. Backend Components:

**API Endpoints:** Test all API endpoints for SQL injection vulnerabilities by attempting to inject SQL code.

**Data Collection:** Ensure that the database connection is secure and that credentials are not hard-coded.

**Authentication and Authorization:** Verify that user authentication mechanisms are robust against SQL injection.

### 7.1.2. Frontend Components:

**Input forms:** Test input forms to ensure they sanitize inputs before sending them to the backend.

**Error Messages:** Ensure that error messages do not reveal sensitive information about the database structure or queries.

**User Feedback:** Verify that users receive appropriate feedback when invalid inputs are detected.

### 7.1.3. Utility Functions:

**Sanitization Functions:** Test functions that sanitize user inputs to ensure they effectively remove or escape SQL injection attempts.

**Logging Functions:** Ensure that logging functions capture relevant information without exposing sensitive data.

**Configuration Functions:** Test functions that manage database configurations to ensure they do not expose sensitive information.

## 7.2. White Box Testing:

**Code Review:** Conduct thorough code reviews to identify potential vulnerabilities in the codebase.

**Static Code Analysis:** Use tools to analyse the code for common SQL injection patterns and vulnerabilities.

**Test Coverage:** Ensure that unit tests cover all critical paths in the code, especially those that handle user inputs and database interactions.



### 7.3. Black Box Testing:

**Penetration Testing-** Simulate real-world attacks to identify vulnerabilities in the application. Focus on SQL injection attempts to assess the system's defences against unauthorized data access.

**Fuzz Testing-** Input random or unexpected data into the application to uncover potential weaknesses. This technique helps identify how the system handles invalid or malicious inputs.

**User Acceptance Testing (UAT)-** Involve end-users in testing to ensure the application meets their requirements and expectations. Validate that the system is secure and functions correctly from a user's perspective.

**Functional Testing-** Verify that all features of the application work as intended without knowledge of the internal code. Test various user scenarios to ensure the application behaves correctly under different conditions.

**Regression Testing-** Ensure that new code changes do not adversely affect existing functionality. Re-run previous test cases to confirm that the application remains secure and stable after updates.

**Performance Testing-** Evaluate the application's responsiveness and stability under various load conditions. Identify how the system performs when subjected to high traffic or simultaneous SQL queries.

## **CHAPTER-8**

### **CONCLUSION**

The outcome of this project is the successful implementation of a system that detects and prevents SQL injection attacks in a web application environment using Python and SQLite. The system utilizes parameterized queries to safeguard against SQL injection by ensuring that user inputs are treated as data, not executable code. Additionally, a detection mechanism based on regular expressions was developed to identify common SQL injection patterns (such as ' OR '1'='1 or UNION SELECT) in SQL queries. The integration of Python's sqlite3 library allows for efficient database operations, while the re library provides an effective way to identify potentially malicious inputs. The system can alert users or administrators whenever a SQL injection attempt is detected, significantly enhancing database security and data integrity.

Conclusion conveys this project successfully demonstrates the integration of Python and SQLite to address the critical issue of SQL injection vulnerabilities. By using parameterized queries through Python's sqlite3 library, the project effectively prevents unauthorized SQL code execution, ensuring secure interaction with the database. The use of Python's re library for pattern detection further enhances the system by identifying potentially harmful SQL injection attempts before they can execute. This dual approach of prevention and detection significantly reduces the risk of data leaks and unauthorized access, making the application more secure.

The lightweight and portable nature of SQLite, combined with Python's simplicity and flexibility, makes this project highly suitable for rapid prototyping and small-scale applications. The solution can be easily extended to more complex systems, incorporating advanced techniques such as machine learning-based anomaly detection or integrating with Web Application Firewalls (WAFs) for real-time protection. Overall, the project highlights the importance of secure coding practices and the effectiveness of using a combination of tools to safeguard against SQL injection attacks.

## 8.2 FUTURE ENHANCEMENT

Looking ahead, future enhancements of this project could involve integrating machine learning models for more advanced and dynamic SQL injection detection. By training models on historical attack data, the system could identify new, previously unseen injection techniques and anomalies in real-time. The integrating a Web Application Firewall to automatically block malicious queries and enhancing logging and alerting mechanisms for better monitoring and reporting would improve the system's robustness.

### Considerations for Future Enhancements

**Scalability:** Ensure that the solution can scale to handle large volumes of data and high-frequency queries in production environments.

**Performance Overhead:** Advanced detection methods, such as real-time monitoring or machine learning, could introduce performance overhead, so optimizing for speed and resource usage will be essential.

**False Positives:** Over time, the detection algorithm should be fine-tuned to minimize false positives, which could block legitimate users or database interactions.

**Security Updates:** Stay updated with the latest attack techniques and SQL injection vectors to adapt the detection system accordingly.

## Sample Coding

```
import sqlite3
import random

def initialize_databases():
    with sqlite3.connect("admin_database.db") as admin_conn:
        admin_cursor = admin_conn.cursor()
        admin_cursor.execute("""
        CREATE TABLE IF NOT EXISTS admin_data (
            id INTEGER PRIMARY KEY,
            username TEXT NOT NULL,
            password TEXT NOT NULL,
            sensitive_info TEXT NOT NULL
        )
        """)
        admin_cursor.executemany("""
        INSERT INTO admin_data (username, password, sensitive_info)
        VALUES (?, ?, ?)
        """, [
            ("admin", "admin123", "Confidential Report A"),
            ("admin", "admin123", "Sensitive Data B"),
        ])
        admin_conn.commit()

    with sqlite3.connect("user_database.db") as user_conn:
        user_cursor = user_conn.cursor()
        user_cursor.execute("""
```

```

CREATE TABLE IF NOT EXISTS user_data (
    id INTEGER PRIMARY KEY,
    username TEXT NOT NULL UNIQUE,
    password TEXT NOT NULL,
    info TEXT
)
")
user_conn.commit()

def log_activity(message, log_file="suspicious_activity.log"):
    with open(log_file, "a") as file:
        file.write(f'{message}\n')

def random_detection():
    if random.choice([True, False]):
        alert = "Random Detection Alert! Potential security breach detected."
        print(alert)
        log_activity(alert)
    else:
        print("No anomalies detected during random check.")

def authenticate(username, password):
    if username == "admin" and password == "admin123":
        return "admin"
    else:
        with sqlite3.connect("user_database.db") as conn:
            cursor = conn.cursor()
            cursor.execute("SELECT * FROM user_data WHERE username = ? AND
password = ?", (username, password))
            user = cursor.fetchone()
            if user:

```

```

        return "user"

    return None

def admin_dashboard():
    print("\n--- Admin Dashboard ---")
    while True:
        print("\n1. Check for Data Leakage")
        print("2. Logout")
        choice = input("Enter your choice: ")
        if choice == "1":
            random_detection()
            username = input("Enter a username to check in admin database: ")
            detect_data_leakage("admin_database.db", "admin_data", username)
        elif choice == "2":
            print("Logging out of Admin Dashboard.")
            break
        else:
            print("Invalid choice. Please try again.")

def detect_data_leakage(database, table, user_input):
    with sqlite3.connect(database) as conn:
        cursor = conn.cursor()
        try:
            query = f"SELECT * FROM {table} WHERE username = '{user_input}'"
            print(f"Executing query: {query}")
            cursor.execute(query)
            results = cursor.fetchall()

            if "OR" in user_input.upper() or ";" in user_input or "--" in user_input or
"UNION" in user_input.upper():
                print("SQL Injection Detected!")

```

```

        log_activity(f'SQL Injection Detected: {query}')
        return

    if len(results) > 1:
        print("Data Leakage Detected!")
        log_activity(f'Data Leakage Detected: {query}')
        return

    print("No data leakage detected. File is safe.")
except sqlite3.Error as e:
    print(f'SQL Error: {e}')
    log_activity(f'SQL Error: {e}')

def user_dashboard(username):
    print(f'\n--- User Dashboard for {username} ---')
    while True:
        print("\n1. Add Data")
        print("2. Logout")
        choice = input("Enter your choice: ")
        if choice == "1":
            add_user_data(username)
        elif choice == "2":
            print("Logging out of User Dashboard.")
            break
        else:
            print("Invalid choice. Please try again.")

def add_user_data(username):
    with sqlite3.connect("user_database.db") as conn:
        cursor = conn.cursor()
        info = input("Enter the information to store: ")
        try:

```



```

        cursor.execute("""
UPDATE user_data SET info = ? WHERE username = ?
""", (info, username))
conn.commit()

        print("Data added successfully.")
    except sqlite3.Error as e:
        print(f"Error adding data: {e}")

def check_if_username_exists(username):
    with sqlite3.connect("user_database.db") as conn:
        cursor = conn.cursor()
        cursor.execute("SELECT username FROM user_data WHERE username = ?",
(username,))
        user = cursor.fetchone()
        if user:
            return True
        return False

def register_new_user():
    print("\n--- Register New User ---")
    username = input("Enter a new username: ")
    password = input("Enter a new password: ")
    if check_if_username_exists(username):
        print("Error: Username already exists. Please try a different one.")
        return
    with sqlite3.connect("user_database.db") as conn:
        cursor = conn.cursor()
        try:
            cursor.execute("INSERT INTO user_data (username, password) VALUES
(?, ?)", (username, password))

```

```

        conn.commit()

        print("New user registered successfully.")
    except sqlite3.IntegrityError:
        print("Error: Username already exists (caught by UNIQUE constraint).
Please try a different one.")
    except sqlite3.Error as e:
        print(f"Error: {e}")

def main():
    initialize_databases()
    print("Databases initialized.")
    while True:
        print("\n--- Main Menu ---")
        print("1. Login")
        print("2. Register New User")
        print("3. Exit")
        choice = input("Enter your choice: ")
        if choice == "1":
            username = input("Enter username: ")
            password = input("Enter password: ")
            user_type = authenticate(username, password)
            if user_type == "admin":
                print("Admin authenticated successfully.")
                admin_dashboard()
            elif user_type == "user":
                print(f"User authenticated successfully. Welcome, {username}.")
                user_dashboard(username)
            else:
                print("Authentication failed. Invalid credentials.")

```

```
elif choice == "2":
    register_new_user()
elif choice == "3":
    print("Exiting program.")
    break
else:
    print("Invalid choice. Please try again.")
if __name__ == "__main__":
    main()
```

## SCREENSHOTS

### 1.ADMIN LOGIN FAILS

```
Python 3.8.10 (tags/v3.8.10:3d8993a, May  3 2021, 11:48:03) [MSC v.1928 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:\Users\nikhi\OneDrive\Desktop\mini project\4.py =====
Databases initialized.

--- Main Menu ---
1. Login
2. Register New User
3. Exit
Enter your choice: 1
Enter username: admin
Enter password: admin321
Authentication failed. Invalid credentials.
```

## 2. ADMIN LOGIN SUCESSFULL

```
--- Main Menu ---
1. Login
2. Register New User
3. Exit
Enter your choice: 1
Enter username: admin
Enter password: admin123
Admin authenticated successfully.

--- Admin Dashboard ---

1. Check for Data Leakage
2. Logout
Enter your choice: 1
No anomalies detected during random check.
Enter a username to check in admin database: admin
Executing query: SELECT * FROM admin_data WHERE username = 'admin'
Data Leakage Detected!
```

## 3.DATA LEAKAGE DETECTION

```
1. Check for Data Leakage
2. Logout
Enter your choice: 1
Random Detection Alert! Potential security breach detected.
Enter a username to check in admin database: user
Executing query: SELECT * FROM admin_data WHERE username = 'user'
No data leakage detected. File is safe.

1. Check for Data Leakage
2. Logout
Enter your choice: 1
Random Detection Alert! Potential security breach detected.
Enter a username to check in admin database: usertwo
Executing query: SELECT * FROM admin_data WHERE username = 'usertwo'
No data leakage detected. File is safe.
```

## 4.USER LOGIN

```
1. Check for Data Leakage
2. Logout
Enter your choice: 2
Logging out of Admin Dashboard.
```

```
--- Main Menu ---
```

```
1. Login
2. Register New User
3. Exit
Enter your choice: 2
```

```
--- Register New User ---
```

```
Enter a new username: shiro
Enter a new password: new_pass123
Error: Username already exists. Please try a different one.
```

## 5. LOG OUT

```
--- Main Menu ---
```

```
1. Login
2. Register New User
3. Exit
Enter your choice: 3
Exiting program.
```

```
>>>
```

## REFERENCES

- [1] A. K. Hegde, and P. N. Jayanthi, "A Survey on SQL Injection Attacks and Prevention Methods",2020.
- [2] U. A. Butt, M. Mehmood, S. B. H. Shah, R. Amin, M. W. Shaukat, S. M. Raza, and M. Piran, "A review of machine learning algorithms for cloud computing security. Electronics," vol. 9, 1379, 2020.
- [3] B. I. Mukhtar, and M. A. Azer, "Evaluating the Modsecurity Web Application Firewall Against SQL Injection Attacks," In 2020 15th International Conference on Computer Engineering and Systems (ICCES), December 2020, pp. 1-6.
- [4] K. Abouelmehdi, H. Khaloufi, and A. Beni hssane, "Risk assessment of SQL injection: An experimental study," In 2021 7th International Conference on Optimization and Applications (ICOA), May 2021, pp. 1-4.
- [5] S. Leelavathy, "A Secure Methodology to Detect and Prevent Ddos and Sql Injection Attacks," Turkish Journal of Computer and Mathematics Education (TURCOMAT), vol.12, 2021, pp.341-346.
- [6] I. Hashem, M. Islam, S. M. Haque, Z. I. Javed, and N. Sakib, "A Proposed Technique for Simultaneously Detecting DDoS and SQL Injection Attacks," International Journal of Computer Applications, vol. 975, pp. 8887.

- [7] S. Alneyadi, E. Sithirasanen, and V. Muthukkumarasamy, Word N-gram based classification for data leakage prevention, in Proc. 12th IEEE Int. Conf. Trust, Secur. Privacy Comput. Commun., Jul. 2013, pp. 578 585.
- [8] S.Xu, BayesianNaïveBayesclassification, J. Inf. Sci., vol. 44, no. 1, pp. 4859, 2016. [
- 9] M. Hart, P. Manadhata, and R. Johnson, Text classification for data loss prevention, in Proc. Int. Symp. Privacy Enhancing Technol. Symp. Berlin, Germany: Springer, 2011, pp. 1837.
- [10] J. Huang, S. Huang, C.-C. Xing, and Y. Qian, Game-theoretic power control mechanismsfordevice-to-devicecommunicationsunderlyingcellular system, IEEE Trans. Veh. Technol., vol. 67, no. 6, pp. 48904900, Jun. 2018.
- [11] X. Shu, J. Zhang, D. D. Yao, and W.-C. Feng, Fast detection of transformed data leaks, IEEE Trans. Inf. Forensics Security, vol. 11, no. 3, pp. 528542, Mar. 2016.
- [12] G. Katz, Y. Elovici, and B. Shapira, CoBAn: A context based model for data leakage prevention, Inf. Sci., vol. 262, pp. 137158, Mar. 2014.



